



Solar ASIC
AUTOMATISATION

Guide d'Installation Complet

Automatisez vos ASIC miners sur energie solaire.

Zéro consommation EDF a la journée | Minage solaire | Bear market proof.

Solaire

Crypto

Home Assistant

Automatise V1.0.37



VERSION

1.0.37

AUTEUR

Halo44

LICENCE

Gratuit & Libre

MATERIEL

Refoss + HA

Sommaire

- 01 Le Concept**
Pourquoi Solar ASIC ? Bear market + solaire
- 02 Architecture**
Vue d'ensemble du systeme complet
- 03 Matériel & Câblage**
Refoss EM06P, prises connectées, placement CT
- 04 Home Assistant**
Intégration et configuration de base
- 05 configuration.yaml**
Capteurs, templates, ASIC prioritaires, revenus
- 06 Automation V1.0.37**
Multi-prioritaire, anti yo-yo, SHED continu
- 07 Bilan quotidien**
Notification 20h fiable côté HA (history_stats)
- 08 Dashboard HTML**
Interface temps reel, graphiques kWh, i18n
- 09 Pools de minage**
7 pools : F2Pool, K1Pool, 2Miners, ViaBTC...
- 10 Acces distant**
Tailscale VPN - accès 4G / WiFi
- 11 Depannage**
FAQ et résolution de problèmes

01 Le Concept

Pourquoi Solar ASIC ?

En bear market crypto, miner avec de l'électricité réseau à 0,20 EUR/kWh revient souvent à miner à perte. La solution : utiliser uniquement l'énergie solaire excédentaire de votre installation. Tant qu'il y a du surplus solaire, les ASIC tournent gratuitement. Dès que le surplus diminue, ils s'éteignent automatiquement pour ne jamais consommer le réseau.

Le cycle journalier automatique

Moment	Comportement
Matin	Lever du soleil -> surplus croissant. Les petits ASIC démarrent un par un (cascade 5s).
Journée	Surplus maximal. L'automation bascule sur les gros ASIC prioritaires (S19, S21...). Minage 100% solaire, zéro EDF.
Soir	Le surplus diminue. Les ASIC s'éteignent progressivement (ordre inverse, LIFO).
Nuit	Tous les ASIC éteints. Aucune consommation réseau.

Différence clef avec la v1 : l'automation V1.0.37 gère une **liste d'ASIC prioritaires**. Le matin, les petits ASIC remplissent le surplus ; dès qu'il y a assez de soleil pour un gros mineur, les petits sont coupés le temps de démarrer le gros, puis reviennent dans le surplus restant. Cascade automatique pour 1, 2, 3 gros ASIC ou plus.

Trois piliers

- **Solaire d'abord** : la production PV est mesurée en temps réel via 6 pinces CT.
- **Décision intelligente** : Home Assistant calcule le surplus et pilote chaque prise.
- **Anti yo-yo** : délais et cooldowns évitent les allumages/extinctions intempestifs dus aux nuages.

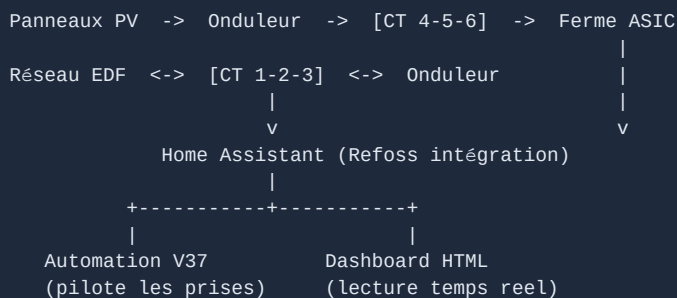
02 Architecture

Vue d'ensemble du système

Cinq maillons relient vos panneaux à vos ASIC. Chacun a un rôle précis dans la chaîne de mesure et de décision.

Maillon	Role
1. Panneaux PV	Produisent l'énergie (ex: 12 kWp)
2. Refoss EM06P	6 pinces CT mesurent réseau (CT 1-2-3) et ferme ASIC (CT 4-5-6)
3. Home Assistant	Cerveau : calcule le surplus, décide quels ASIC allumer (Raspberry Pi)
4. Prises connectées	Executent : WiFi / Zigbee / Matter (Tuya, Shelly...)
5. ASIC Miners	Minent : IceRiver, Goldshell, Antminer...

Flux de données



Atout	Detail
Automatique 24h/24	L'automation tourne en continu, cycle toutes les 5 secondes
Anti yo-yo	Cooldown 30s + boot prioritaire 10 min
Accès distant	Tailscale VPN pour consulter en 4G
Coût EDF	0 EUR pour miner à la journée

03 Matériel & Câblage

Schéma de câblage - pinces CT Refoss

Le Refoss EM06P possède 6 canaux de mesure (6 pinces ampère-métriques). La répartition est cruciale pour que les calculs de surplus soient justes.

Pinces	Emplacement	Mesure
CT 1, 2, 3	Câbles EDF, entre le Linky et l'onduleur	Mesure import / export réseau (signe : + import, -injection)
CT 4, 5, 6	Câbles de sortie onduleur (cote AC)	Mesure la consommation de la ferme ASIC (toujours positif)

IMPORTANT : le câble de tension du Refoss (L1 / L2 / L3 / N) doit être branché sur le même circuit que les CT 1-2-3, sinon les mesures de puissance seront fausses.

Liste de matériel

Element	Modèle conseillé	Usage
Compteur CT 6 canaux	Refoss EM06P (ou 2x Shelly 3EM)	Mesure solaire + réseau
Prises connectées	Tuya / Shelly / Matter, 1 par ASIC	Allumage/extinction pilote
Mini-PC / Pi	Raspberry Pi 3B+ minimum ou PC	Héberge Home Assistant
ASIC miners	IceRiver, Goldshell, Antminer...	Au choix selon budget

Astuce prises : vérifiez l'ampérage max de chaque prise. Un Antminer S19 (~3400W) demande une prise 16A dédiée. Les petits IceRiver (~100W) passent sur des prises 10A standard.

04 Home Assistant

Configuration de base

Avant de coder les capteurs, intégrez le matériel et préparez l'environnement HA.

Etapas préliminaires

- 1. Intégration Refoss** : Paramètres -> Appareils et services -> Ajouter -> Refoss. Les 6 capteurs `sensor.em06_channel_X_power` apparaissent.
- 2. Intégration des prises** : ajoutez vos prises (Tuya, Shelly...). Notez leurs `entity_id` (Outils dev -> Etats -> filtrer 'switch.').
- 3. Token longue durée** : Profil -> Sécurité -> Créer un jeton. Copiez-le pour le dashboard.
- 4. Dossier www** : créez `/config/www/` s'il n'existe pas (pour héberger le dashboard).

CORS : pour que le dashboard HTML accède à l'API HA, ajoutez le bloc `http:` avec `cors_allowed_origins` dans `configuration.yaml` (voir section 05).

Recorder (historique)

Le dashboard affiche des graphiques jour/semaine/mois. Pour cela, HA doit conserver l'historique des capteurs pendant au moins 31 jours :

```
recorder:
  purge_keep_days: 31
  commit_interval: 60
  include:
    entities:
      - sensor.production_solaire
      - sensor.consomption_ferme
      - sensor.consomption_reseau
      - sensor.injection_reseau
      - sensor.asic_allumes_count
```

05 configuration.yaml

Capteurs calculés (templates Jinja2)

Ces capteurs transforment les mesures brutes des CT en valeurs exploitables : production solaire, surplus, import/injection réseau, comptage ASIC.

Ce que vous devez adapter

- **sensor.em06_channel_X_power** -> les noms réels de vos capteurs Refoss
- **switch.asic_small_1 ...** -> les entity_id de vos prises
- **100, 360, 3400** -> la consommation réelle (W) de chaque ASIC

```
template:
- sensor:
  - name: "reseau_total"          # CT 1-2-3 (+ import / - injection)
    unit_of_measurement: "W"
    state: >
      {% set p1 = states('sensor.em06_channel_1_power')|float(0) %}
      {% set p2 = states('sensor.em06_channel_2_power')|float(0) %}
      {% set p3 = states('sensor.em06_channel_3_power')|float(0) %}
      {{ (p1 + p2 + p3) | round(1) }}

  - name: "injection_reseau"      # surplus vers EDF
    state: >
      {% set p = states('sensor.reseau_total')|float(0) %}
      {{ (p * -1) if p < 0 else 0 }}

  - name: "consommation_ferme"    # CT 4-5-6 (conso ASIC)
    state: >
      {% set p4 = states('sensor.em06_channel_4_power')|float(0) %}
      {% set p5 = states('sensor.em06_channel_5_power')|float(0) %}
      {% set p6 = states('sensor.em06_channel_6_power')|float(0) %}
      {{ [p4 + p5 + p6, 0] | max | round(1) }}

  - name: "production_solaire"    # ferme + injection
    state: >
      {% set f = states('sensor.consommation_ferme')|float(0) %}
      {% set i = states('sensor.injection_reseau')|float(0) %}
      {{ (f + i) | round(1) }}
```

ASIC prioritaires (source de vérité)

Le cœur du système V37 : une seule liste définit quels ASIC sont prioritaires (les gros mineurs). L'automation lit cette liste et gère tout automatiquement. Pour ajouter un ASIC (ex: un nouveau S21), il suffit d'ajouter une ligne ici - aucune modification de l'automation.

```
- name: "asic_prio_config"
  state: "ok"
  attributes:
    asics: >-
      {{ [
        {'entity': 'switch.asic_big_1', 'power': 3400, 'name': 'S19'}
      ] }}
  # Plusieurs prioritaires :
  # {'entity': 'switch.s19', 'power': 3450, 'name': 'S19'},
  # {'entity': 'switch.s21_1', 'power': 3500, 'name': 'S21 #1'},
  # {'entity': 'switch.s21_2', 'power': 3500, 'name': 'S21 #2'}
```

Cinq capteurs dérivés (à copier tels quels) calculent automatiquement : la puissance nécessaire au prochain démarrage, le prochain ASIC à allumer, le dernier allumé, et le compteur d'ASIC prioritaires actifs.

Capteur	Rôle
<code>asic_prio_power_needed</code>	Puissance totale requise pour démarrer le prochain prioritaire
<code>asic_prio_next_entity</code>	entity_id du prochain ASIC à allumer (FIFO)
<code>asic_prio_last_entity</code>	entity_id du dernier allumé (premier à couper, LIFO)
<code>asic_prio_count_on</code>	Nombre d'ASIC prioritaires actuellement ON
<code>binary_sensor.asic_prio_add_ready</code>	ON quand la prod solaire couvre le prochain prioritaire

Revenus minage en euros

Deux capteurs template multiplient la production de chaque pool (en crypto) par le prix en euros récupéré via CoinGecko, pour afficher les revenus réels en EUR.

```
- platform: rest          # prix crypto, 1 appel / 10 min
  name: "crypto_prices_eur"
  resource: "https://api.coingecko.com/api/v3/simple/price
    ?ids=bitcoin,kaspa,alephium&vs_currencies=eur"
  scan_interval: 600
  json_attributes: [bitcoin, kaspa, alephium]
```


06

Automatisation V1.0.37

Logique multi-prioritaire

L'automation V1.0.37 tourne en mode single (une seule instance a la fois) avec 5 déclencheurs. Elle ne contient aucun nom d'ASIC en dur : tout vient de sensor.asic_prio_config.

Déclencheur	Condition	Action
cycle_normal	Toutes les 5s	Régulation continue : ajoute/retire des petits ASIC
maybe_prio_add	Prod >= besoin, 1 min	Démarre le prochain ASIC prioritaire
sustained_import	Grid > 250W, 5 min	Réduit les petits ASIC
production_drop	Prod < 2800W, 5 min	Coupe le dernier prioritaire (soir)
endofday_cleanup	Grid > 50W, 10 min	Coupe tous les petits restants (nuit)

Séquence de démarrage d'un prioritaire

Quand la production solaire couvre la puissance du prochain gros ASIC pendant 1 minute :

- Tous les petits ASIC sont coupes (libération du bus 230V)
- Après 25s, le gros ASIC démarre
- Pendant 10 min, l'ajout de petits ASIC est **bloqué** (le gros monte en puissance ~5 min)
- Une fois stabilise, les petits reviennent dans le surplus restant

Pourquoi le blocage 10 min ? Un gros ASIC (S19) ne consomme pas ses 3400W instantanément : il met ~5 min a monter en charge. Sans blocage, le cycle ADD verrait un faux surplus, allumerait des petits ASIC, puis devrait les recouper quand le gros atteint sa pleine puissance -> import EDF et yo-yo. La variable prio_boot_ok empêche ça.

SHED continu - la correction clef

Le déclencheur sustained_import ne se déclenche qu'une fois. Pour réagir a un import qui persiste, le cycle_normal intégré une branche SHED : si grid > 50W et qu'il reste des petits ASIC, il en coupe un toutes les 5s (avec cooldown 30s). Cela ramène la consommation sous la production progressivement, sans yo-yo.

```
# Dans cycle_normal :
- choose:
  - conditions:
    - "{{ grid_total > 50 and nb_small_on > 0
      and switch_cooldown_ok }}"
  sequence:
    - action: switch.turn_off
      target: {entity_id: "{{ small_on[-1] }}"}
```

07 Bilan quotidien

Notification fiable côté Home Assistant

Chaque jour à 20h, une automation HA envoie un bilan complet sur Telegram, ntfy et l'app HA Companion. Contrairement à une version navigateur, elle tourne 24/7 : les durées de minage sont donc toujours exactes, même si le dashboard est fermé.

Capteurs history_stats

Home Assistant calcule en continu le temps ON de chaque ASIC (reset à minuit). Ces capteurs alimentent le bilan avec des durées fiables.

```
sensor:
  - platform: history_stats
    name: "asic_big_1_heures_jour"
    entity_id: switch.asic_big_1
    state: "on"
    type: time
    start: "{{ today_at('00:00') }}"
    end: "{{ now() }}"
```

Contenu du bilan

Bloc	Detail
Energie	Solaire produit, conso ASIC, import/injection EDF (kWh)
Durées	Minage total + heures par ASIC prioritaire (history_stats)
Revenus	Revenus minage de la veille (J-1) en euros
Net	Revenus - cout EDF (l'injection n'est pas comptée si pas de contrat rachat)

Net sans injection : sans contrat de rachat EDF, l'énergie injectée n'est pas rémunérée. Le calcul net = revenus minage - cout import EDF. L'injection reste affichée à titre indicatif.

Canaux de notification

- **Telegram** : via un bot (token + chat_id), message instantané
- **ntfy** : push open-source, topic personnalisée (app ntfy.sh)
- **HA Companion** : notification native sur l'app mobile Home Assistant

08 Dashboard HTML

Interface visuelle temps réel

Un fichier HTML unique (HTML + CSS + JS) a déposer dans /config/www/. Il lit l'API HA et affiche production, consommation, flux énergie anime, revenus minage et historique.

#	Etape	Detail
1	Copier le fichier	/config/www/dashboard_mining.html
2	Copier secrets.example.js	-> secrets.js (vos tokens et IDs)
3	Créer un token API	Profil HA -> Securite -> Jetons longue duree
4	Ouvrir le dashboard	http://IP_HA:8123/local/dashboard_mining.html

Variables a personnaliser (secrets.js)

Variable	Default	Remplacer par
HA_URL_LOCAL	http://192.168.1.XX:8123	IP locale de votre HA
HA_URL_VPN	http://100.X.X.X:8123	IP Tailscale (optionnel)
HA_TOKEN	eyJhbGci...	Votre token API HA
MINING_COINS	[...]	Liste de vos cryptos minées
F2POOL_TOKEN	votre_token	Clé API du pool

Fonctionnalités du dashboard

- **Flux énergie anime** : solaire -> ferme -> ASIC, avec import/injection EDF
- **Graphique jour** : courbes de puissance (W) sur 24h
- **Graphique semaine/mois** : barres kWh quotidiennes
- **Revenus minage** : par crypto, en temps réel et en euros
- **Bilan hier** : revenus - cout EDF de la veille
- **Multi-langue**: français, anglais, allemand, espagnol
- **Prochain démarrage** : chrono et seuil du prochain ASIC

Graphiques en barres : pour les vues semaine et mois, le dashboard intègre la puissance (W) en énergie (kWh) par jour via la méthode trapézoïdale, puis affiche 4 barres par jour (solaire, ferme, import, injection).

09 Pools de minage

7 pools supportés

Solar ASIC peut afficher les revenus de n'importe quel pool. L'architecture est universelle : Home Assistant récupère les données cote serveur (pas de problème CORS), le dashboard lit les capteurs HA. Tous les capteurs doivent être nommés `sensor.f2pool_SYMBOLE_day` et `sensor.f2pool_SYMBOLE_unpaid` (le préfixe `f2pool_` est conventionnel, quel que soit le pool réel).

Pool	Cryptos	Auth	Type	Facilité
F2Pool	BTC, ALPH, KAS, LTC...	Token V2 (POST)	command_line	Difficile
K1Pool	KAS, RXD, ZIL...	Wallet (GET)	command_line	Facile
2Miners	ETC, RVN, KAS, ZEC...	Wallet (GET)	rest:	Facile
ViaBTC	BTC, ETH, LTC...	API Key (header)	rest:	Moyen
NiceHash	BTC (tous algos)	HMAC complexe	command_line	Difficile
HeroMiners	KAS, ALPH, RXD...	Wallet (GET)	rest:	Facile
Kryptex	BTC, KAS, RXD...	Token (GET)	rest:	Moyen

Demarche universelle (toute nouvelle pool)

1. Tester l'API depuis PowerShell (Invoke-RestMethod) et noter la structure JSON
2. Créer un capteur HA : rest: si GET simple, command_line: si POST ou auth complexe
3. Créer des template sensors `f2pool_SYMBOLE_day` et `f2pool_SYMBOLE_unpaid`
4. Ajouter la crypto dans `MINING_COINS` de `secrets.js`
5. Redémarrer HA et vérifier dans Outils dev -> Etats

Exemple 2Miners (GET public, wallet)

API publique sans authentification : l'adresse wallet suffit. Sous-domaine par crypto (etc.2miners.com, kas.2miners.com...). Retourne balance, récompense 24h et workers.

```
rest:
- resource: "https://etc.2miners.com/api/accounts/TON_WALLET"
  scan_interval: 300
  sensor:
    - name: "2miners_etc_raw"
      value_template: >
        {{ (value_json.stats.balance|float(0) / 1e9) | round(6) }}
      json_attributes: [stats, "24hreward", workersOnline]

template:
- sensor:
    - name: "f2pool_etc_day"      # nom generique dashboard
      state: >
        {% set r = state_attr('sensor.2miners_etc_raw', '24hreward') %}
        {{ (r|float(0) / 1e9) | round(6) if r else 0 }}
```

Diviseur selon le coin : ETC = / 1 000 000 000, KAS = / 100 000 000, ZEC = / 100 000 000. Vérifiez toujours la réponse brute avant de choisir le diviseur.

Exemple ViaBTC (GET + API Key header)

Clé API dans le header (pas de signature HMAC). Endpoints séparés pour le hashrate et les profits.

```
rest:
- resource: "https://www.viabtc.com/res/openapi/v1/profit?coin=BTC"
  scan_interval: 300
  headers:
    Authorization: "TON_API_KEY_VIABTC"
  sensor:
    - name: "viabtc_btc_profit"
      value_template: "{{ value_json.data.total_profit|float(0) }}"
```

Exemple NiceHash (HMAC - script Python)

NiceHash utilise une authentification HMAC complexe : chaque requête est signée avec timestamp + nonce. Impossible directement avec les capteurs HA -> on passe par un script Python sur le Pi, appelle en command_line.

```
# /config/scripts/nicehash_balance.py
import hashlib, hmac, uuid, time, requests
API_KEY='...'; API_SECRET='...'; ORG_ID='...'
def sign(method, path, ts, nonce, body=''):
    msg = f'{API_KEY}\x00{ts}\x00{nonce}\x00\x00{ORG_ID}'
    msg += f'\x00\x00{method}\x00{path}\x00{body}'
    return hmac.new(API_SECRET.encode(), msg.encode(),
                    hashlib.sha256).hexdigest()
ts=str(int(time.time()*1000)); nonce=str(uuid.uuid4())
path='/main/api/v2/accounting/account2/BTC'
sig=sign('GET', path, ts, nonce)
headers={'X-Time':ts, 'X-Nonce':nonce, 'X-Organization-Id':ORG_ID,
        'X-Auth':f'{API_KEY}:{sig}'}
r=requests.get(f'https://api2.nicehash.com{path}', headers=headers)
print(r.json().get('totalBalance',{}).get('available','0'))
```

```
# configuration.yaml
command_line:
  - sensor:
      name: "nicehash_btc_balance"
      scan_interval: 300
      command: "python3 /config/scripts/nicehash_balance.py"
      value_template: "{{ value | float(0) }}"
      command_timeout: 20
```

Endpoints des autres pools

Pool	Endpoint	Champs clefs
K1Pool	GET /api/miner/{coin}/{wallet}	miner.pendingBalance, miner.coinsPerDay
HeroMiners	GET /api/stats_address?address={w}	stats.balance, stats.paid_today
Kryptex	GET /api/v1/hashrates?token={t}	data.balance, data.todayIncome
F2Pool	POST /v2/assets/balance	balance, yesterday_income

10 Accès distant

Tailscale VPN

Tailscale crée un réseau privé entre vos appareils. Vous accédez à votre dashboard depuis n'importe où (4G, WiFi extérieur) comme si vous étiez chez vous, sans ouvrir de port.

#	Etape	Detail
1	Add-on HA	Paramètres -> Add-ons -> Tailscale -> Démarrer
2	Compte Tailscale	tailscale.com (gratuit) -> noter l'IP 100.x.x.x
3	App mobile	Installer Tailscale iOS/Android, même compte
4	Modifier secrets.js	Renseigner HA_URL_VPN avec votre IP 100.x.x.x

```
const HA_URL_LOCAL = 'http://192.168.1.XX:8123';
const HA_URL_VPN   = 'http://100.X.X.X:8123';
// Bascule auto selon le reseau detecte
let HA_URL = location.hostname.startsWith('192.168')
  ? HA_URL_LOCAL : HA_URL_VPN;
```

Bascule automatique : le dashboard détecte s'il est ouvert en local (192.168.x.x) ou à distance, et choisit l'URL adaptée. Transparent pour l'utilisateur.

11 Dépannage

FAQ et résolution de problèmes

Problème	Solution
Dashboard affiche ERREUR	Token invalide ou IP incorrecte. Recréez le token dans HA et vérifiez l'URL dans secrets.js.
Capteurs unavailable	Rechargez les templates : Outils dev -> YAML -> Entites basées sur modèle.
ASIC ne s'allument pas	Vérifiez que l'automation V1.0.37 est active et que le surplus dépasse le seuil pendant la durée requise.
IDs de prises changes	Outils dev -> Etats -> filtrer 'switch.' pour retrouver les nouveaux entity_id.
Graphique jour vide	Normal les premières 24h. L'historique s'accumule progressivement.
Graphique semaine/mois vide	Vérifiez recorder: purge_keep_days >= 31 et que les capteurs sont bien dans include:.
Import EDF malgré le surplus	Le filtre ignore l'import < 20W (bruit CT Refoss). Au-delà vérifiez le calibrage des pinces.
Prix crypto a unknown	Attendez 10 min (1er appel CoinGecko) et vérifiez l'accès internet du Pi.
Bilan 20h non reçu	Vérifiez les credentials Telegram/ntfy et que l'automation bilan est active. Testez-la manuellement.
Yo-yo des ASIC	Augmentez le cooldown (30s) ou le délai boot prioritaire (10 min) dans l'automation.
Inaccessible en 4G	Tailscale doit être actif sur le téléphone ET sur HA. Vérifiez l'IP 100.x.x.x dans l'app.

12 Bonne installation !

Ce projet est partagé gratuitement. Améliorez-le, distribuez-le, partagez-le avec la communauté.

Checklist finale

- [V] Refoss EM06P intègre dans HA (6 capteurs de puissance visibles)
- [V] configuration.yaml avec templates, ASIC prioritaires et revenus
- [V] Automation V37 active (multi-prioritaire, SHED, anti yo-yo)
- [V] Automation bilan quotidien 20h configurée (Telegram/ntfy/HA)
- [V] Dashboard HTML personnalisée dans /config/www/
- [V] secrets.js rempli (jamais commite sur GitHub)
- [V] Accès distant Tailscale (optionnel)

Solar ASIC v4 - Halo44 - Licence libre.

Minions propre. Code source et exemples disponibles sur GitHub.